Reflection Log

Noorinder

Credit 2
Assignment Name: Word Count

How has your program changed from planning to coding to now?

At first, the program was planned to count the number of words and calculate the average word length in a file named `source.txt`. The idea was to open the file, read its content line by line, and process each word to calculate the desired metrics. However, during the implementation and testing phases, issues arose that required adjustments to improve functionality and reliability.

1. Incorrect File Path

- **Problem:** The program threw a `FileNotFoundException` because the file path specified in the code was incorrect. The initial path `"source.txt"` didn't work as it didn't point to the correct location of the file.
- **Fix:** I replaced the file path with a valid relative path pointing to the file's location within the project folder structure. This ensured the program could locate the file without errors.

Before Code:

```
String fileName = "source.txt"; // Incorrect file path
BufferedReader br = new BufferedReader(new FileReader(fileName));
```

After Fix:

```
String fileName = "../Chapter11/src/Mastery/source.txt"; // Corrected relative path
BufferedReader br = new BufferedReader(new FileReader(fileName));
```

2. Missing Braces for Try Block

- **Problem:** The program failed to compile because the `try` block was missing curly braces `{}`. Java requires braces around `try` blocks to define their scope, even if they only contain one line of code.
- **Fix:** I added curly braces to properly enclose all the statements inside the `try` block.

Before Code:

```
try
BufferedReader br = new BufferedReader(new FileReader(fileName)); // Syntax error
```

After Fix:

```
try {
    BufferedReader br = new BufferedReader(new FileReader(fileName));
}
```

3. Logic Error in Word Counting

- **Problem:** The program didn't count any words because of a logic error in the word-checking condition. I had written `if (word.isEmpty())`, which caused the program to skip all valid words, resulting in `wordCount` remaining zero.
- **Fix:** I updated the condition to `if (!word.isEmpty())`, ensuring valid words were counted and processed correctly.

Before Code:

```
if (word.isEmpty()) {
    wordCount++;
    totalLength += word.length();
}
```

After Fix:

```
if (!word.isEmpty()) { // Correct condition to count valid words
    wordCount++;
    totalLength += word.length();
}
```

4. Unclear Exception Handling

- **Problem:** When an error occurred while reading the file, the program displayed a generic error message (“An error occurred”) without providing specific details. This made debugging difficult as the message didn’t indicate what went wrong.
- **Fix:** I replaced the generic message with detailed output, including the exception type and suggestions for the user to check the file path and permissions. This made the program easier to debug and more user-friendly.

Before Code:

```
catch (IOException e) {
    System.out.println("An error occurred.");
}
```

After Fix:

```
catch (IOException e) {
    System.out.println("An error occurred while reading the file. Please check the file path and permissions.");
    e.printStackTrace(); // Includes the stack trace for debugging
}
```